

آموزش زبان C

تهیه و تألیف: محمد مهدی قائمی

Mahdighaemi123@gmail.com

Codewithmahdi.ir

@codewithmahdi

V1.0.0

فهرست مطالب

فصل ۱: مقدمات، نصب و انواع داده‌ها	۵
۱. برنامه‌نویسی C چیست؟ چرا C؟	۵
۲. نصب و راه‌اندازی (آماده‌سازی محیط کار)	۶
۳. تشریح ساختار برنامه (کد بالا چه کار می‌کند؟)	۷
۴. متغیرها و انواع داده‌ها	۸
هشدار مهم: مقدار زباله (Garbage Value)	۹
فصل ۲: عملگرها و ساختارهای کنترلی	۱۰
۱. عملگرهای ریاضی و انتساب یا همان ابزارهای محاسباتی	۱۰
۲. عملگرهای مقایسه‌ای و منطقی یا همان تصمیم‌گیری	۱۲
۳. ساختارهای شرطی if و switch	۱۲
۴. حلقه‌های تکرار یا Loops	۱۵

۵. کنترل پیشرفته حلقه با break و continue ۱۶

فصل ۳: توابع یا Functions و مدولار سازی ۱۷

۱. تعریف تابع ۱۷

۲. ارسال پارامتر به روش Call by Value ۱۸

۳. پروتوتایپ یا Prototype ۲۰

۴. تقسیم پروژه به چندین فایل (include, Header) ۲۰

فصل ۴: آرایه‌ها و رشته‌ها یا Arrays & Strings ۲۳

۱. آرایه‌های یک‌بعدی ۲۳

۲. آرایه‌های چندبعدی ۲۵

۳. رشته‌ها در C یا C Strings ۲۵

۴. توابع کتابخانه‌ای رشته در string.h ۲۶

فصل ۵: اشاره‌گرها یا Pointers و استراکچر یا Structs ۲۷

۱. اشاره‌گر چیست؟ ۲۷

۲. استراکچر یا Struct ۲۸

۳. ارسال با مرجع یا Call by Reference ۲۹

۴. اشاره‌گر به استراکچر ۳۰

فصل ۶: دریافت ورودی و تبدیل انواع داده ۳۱

۱. دریافت ورودی با scanf ۳۱

۲. مثال‌های کاربردی دریافت ورودی ۳۲

۳. استثنای رشته‌ها (String) ۳۴

فصل ۷: ۲۰ مثال کاربردی و کوتاه ۳۷

۱. سلام دنیا ۳۷

۲. دریافت عدد از کاربر ۳۷

۳۷	۳. جمع دو عدد صحیح
۳۸	۴. بررسی زوج یا فرد بودن
۳۸	۵. تبدیل دما از سلسیوس به فارنهایت
۳۸	۶. پیدا کردن بزرگترین عدد بین سه متغیر
۳۸	۷. چاپ ایام هفته با Switch
۳۸	۸. چاپ اعداد ۱ تا ۱۰
۳۹	۹. محاسبه فاکتوریل
۳۹	۱۰. جدول ضرب ۵
۳۹	۱۱. جمع عناصر آرایه
۳۹	۱۲. پیدا کردن ماکسیمم در آرایه
۴۰	۱۳. شمارش حروف یک رشته
۴۰	۱۴. کپی کردن رشته
۴۰	۱۵. تابع محاسبه توان
۴۰	۱۶. کار با اشاره گر ساده
۴۰	۱۷. جابجایی دو عدد با اشاره گر یا Swap
۴۱	۱۸. تعریف استراکچر دانشجو
۴۱	۱۹. چاپ کد اسکی کاراکتر
۴۱	۲۰. ماشین حساب ساده

فصل ۸: ۲۰ تمرین ۴۲

۴۲	۱. چاپ هرم ستاره‌ای
۴۳	۲. مثلث اعداد
۴۳	۳. جدول ضرب
۴۴	۴. تشخیص عدد اول
۴۴	۵. سری فیبوناچی
۴۵	۶. معکوس یا برعکس کردن یک عدد

۴۵	۷. ادغام دو آرایه یا Merge Arrays
۴۶	۸. محاسبه فاکتوریل با بازگشت
۴۶	۹. حذف عناصر تکراری از آرایه
۴۷	۱۰. بزرگترین مقسوم علیه مشترک یا BMM
۴۷	۱۱. جستجوی خطی یا Linear Search
۴۸	۱۲. جستجوی دودویی یا Binary Search
۴۹	۱۳. مرتب سازی حبابی یا Bubble Sort
۴۹	۱۴. جمع دو ماتریس
۵۰	۱۵. محاسبه میانگین عناصر آرایه
۵۰	۱۶. شمارش تعداد حروف صدا دار
۵۱	۱۷. چرخش آرایه یا Array Rotation
۵۱	۱۸. تبدیل حروف کوچک به بزرگ
۵۲	۱۹. پیدا کردن عنصر دوم بزرگ آرایه
۵۳	۲۰. بررسی پالیندروم بودن رشته

فصل ۱: مقدمات، نصب و انواع داده‌ها

به دنیای برنامه‌نویسی خوش آمدید. در این فصل قرار است با زبان قدرتمند C آشنا شویم، ابزارهای لازم را نصب کنیم و یاد بگیریم چطور اولین دستورات را در حافظه کامپیوتر بنویسیم.

۱. برنامه‌نویسی C چیست؟ چرا؟

زبان C اغلب به عنوان «زبان برنامه‌نویسان حرفه‌ای» شناخته می‌شود. این زبان طوری طراحی شده که کمترین مانع را سر راه شما قرار دهد و به شما اجازه دهد دقیقاً همان کاری را که می‌خواهید با سخت‌افزار انجام دهید.

ویژگی‌های کلیدی C

- **سریع ولی حساس**: برنامه‌های C بسیار سریع اجرا می‌شوند، اما این زبان «بخشنده» نیست. اگر اشتباه کنید، معمولاً به جای دریافت یک پیام خطای مؤدبانه، برنامه می‌بندد Crash می‌کند.
- **مدیریت دستی حافظه**: برخلاف زبان‌هایی مثل جاوا یا پایتون، در C چیزی به نام زباله‌روب (Garbage Collector) وجود ندارد. یعنی اگر حافظه‌ای را گرفتید، خودتان مسئول پاک کردن آن هستید. این ویژگی C را برای سیستم‌عامل‌ها و کارهای سنگین عالی می‌کند، اما مسئولیت برنامه‌نویس را بالا می‌برد.

- **سادگی :** تعداد کلمات کلیدی و دستورات این زبان کم است و اگر دو ویژگی کار مشابهی انجام دهند، C فقط یکی را نگه می‌دارد.

۲. نصب و راه‌اندازی (آماده‌سازی محیط کار)

برای اینکه کدهای ما به زبان ماشین (صفر و یک) و قابل اجرا در سیستم تبدیل شود، به ابزاری به نام **کامپایلر (Compiler)** نیاز داریم. معروف‌ترین کامپایلر برای این زبان GCC نام دارد.

نصب کامپایلر:

- **در ویندوز:** ساده‌ترین راه نصب **MinGW** است. (در گوگل جستجو کنید آموزش

نصب MinGW آموزش تصویری خوب هستش)

- **در لینوکس/مک:** ترمینال را باز کنید و دستور `sudo apt install gcc` را بنزید.

اولین برنامه شما (Hello World): یک فایل متنی جدید باز کنید و در هنگام ذخیره `save as` نام آن را `hello.c` بگذارید (پسوند `c`. مهم است).

البته پیشنهاد می‌کنم `vs code` رو نصب کنید که خیلی بهتر میتونید داخلش کد بنزید

کد زیر را دقیقاً در آن بنویسید (بدون شماره سطر) :

```
1. #include <stdio.h>
2.
3. int main() {
4.     printf("Hello World\n");
5.     return 0;
6. }
```

اجرای برنامه : ترمینال یا CMD را در محل فایل باز کنید و دستورات زیر را وارد کنید

(شیفت رو نگه دارید و در پوشه ای که کدتون هست کلیک راست کنید و `open terminal`

یا `cmd` رو انتخاب کنید)

۱. کامپایل کردن `gcc hello.c -o hello` :/این دستور فایل `hello.c` را می‌خواند و

یک فایل اجرایی به نام `hello` می‌سازد.

۲. اجرا کردن:

○ در ویندوز `hello` :

○ در لینوکس یا مک `./hello` :

اگر پیام `Hello World` را دیدید، تبریک می‌گوییم! شما اولین برنامه خود را نوشتید.

۳. تشریح ساختار برنامه (کد بالا چه کار می‌کند؟)

بیایید کدی که نوشتیم را خط به خط بررسی کنیم. هر برنامه C باید حتماً یک نقطه شروع داشته باشد:

۱. `#include <stdio.h>` این خط به کامپایلر می‌گوید که محتویات فایل `stdio.h`

را به برنامه اضافه کن. این فایل شامل ابزارهای ورودی و خروجی استاندارد

(Standard Input/Output) است. بدون این خط، کامپایلر نمی‌داند `printf`

چیست. (اینجوری از کدها و ابزارهایی که قبلاً در زبان C وجود داشتن میتونیم استفاده کنیم)

۲. `int main()` اجرای تمام برنامه‌های C دقیقاً از تابعی به نام `main` شروع می‌شود.

پرانتر () نشان می‌دهد که `main` یک تابع است و آکولاد باز { شروع بدنه

دستورات را مشخص می‌کند. (در فصل‌های بعد درمورد تابع بیشتر صحبت

میکنیم الان صرفاً قبول کنید که محل اجرای دستورات همین تابع `main` هست)

۳. `printf("Hello World\n");` این دستور متن را در خروجی چاپ می‌کند. آن

`\n` در انتهای متن به معنی "New Line" است و باعث می‌شود بعد از چاپ، به

خط بعدی برود.

- نکته: تمام دستورات در C باید با نقطه-ویرگول (;) تمام شوند. فراموش کردن آن یک خطای رایج است. (و باعث ارور میشه پس یادتون نره)
- ۴. `return 0;` این دستور پایان تابع `main` را اعلام می‌کند. بازگرداندن عدد 0 یک قرارداد است که به سیستم‌عامل می‌گوید برنامه با موفقیت و بدون خطا تمام شد.

۴. متغیرها و انواع داده‌ها

متغیر (Variable) در واقع یک ظرف یا جعبه نام‌گذاری شده در حافظه است که می‌تواند یک مقدار را در خود نگه دارد. قبل از استفاده از هر متغیر، باید نوع و نام آن را به کامپایلر بگویید (بهش می‌گن `Declare` کردن متغیر) کنید.

یک راه عالی برای تصور متغیرها این است: یک جعبه بکشید و مقدار فعلی را داخل جعبه بنویسید. (شما می‌توانید مقدار جعبه رو بخونید و یا ویرایش یا حذف کنید)

حالا تو زبان C ما باید مشخص کنید که داخل این جعبه ها چه جنس داده ای قرار می‌گیره مثلاً متنه یا عدد:

انواع داده اصلی (Primitive Types)

زبان C مجموعه‌ای کوچک و استاندارد از انواع داده دارد:

۱. `char` کاراکتر:

- برای ذخیره یک حرف یا علامت مثل 'A' یا 'z'
- اندازه آن معمولاً ۱ بایت (۸ بیت) است.
- مثال C:

مقدار حتماً بین تک کوتیشن باشه // `char letter = 'B';` 1.

۲. `int` عدد صحیح:

- برای ذخیره اعداد بدون اعشار (مثل ۱۲، ۱۰۰، -۴۲).
- اندازه آن معمولاً ۳۲ بیت (۴ بایت) است.
- مثال:

```
1. int count = 10;
2. int age; // تعریف بدون مقداردهی اولیه
```

۳. float و double اعداد اعشاری:

- برای اعداد دارای ممیز (مثل ۳.۱۴).
- double دقت بیشتری نسبت به float دارد (حدود ۱۵ رقم دقت در مقابل ۶ رقم) و در برنامه‌های C رایج‌تر است.
- مثال:

```
1. double pi = 3.14159;
2. float height = 1.75f;
```

مثال: چاپ مشخصات یک کالا (Item Info)

```
1. #include <stdio.h>
2.
3. int main() {
4.     // Defining variables for item details
5.     int count = 5;
6.     float price = 12.50;
7.     char category = 'A';
8.
9.     // Printing all types in one line
10.    printf("Count: %d, Price: %.2f, Category: %c\n", count, price,
category);
11.
12.    return 0;
13. }
```

هشدار مهم: مقدار زباله (Garbage Value)

در C، وقتی متغیری را تعریف می‌کنید محتوای آن به صورت خودکار صفر یا پاک نمی‌شود. این متغیر حاوی یک مقدار تصادفی و بی‌معنی است که از قبل در آن بخش از حافظه وجود داشته (به آن مقدار زباله می‌گویند). بنابراین، همیشه قبل از استفاده از متغیر، به آن مقدار بدهید.

```
1. int x;
2. // اگه الان مقدار ایکس رو بخونید ممکنه منفی هزار باشه
3. x = 5; // حالا ایکس مقدار معتبر داره
```

نکات تکمیلی:

- زبان C به بزرگی و کوچکی حروف حساس است. (Case Sensitive) متغیر num با Num فرق دارد.
- می‌توانید چند متغیر هم‌نوع را در یک خط تعریف کنید

```
1. int x, y, z;
```

فصل ۲: عملگرها و ساختارهای کنترلی

. حالا که در فصل اول یاد گرفتیم چطور داده‌ها را در متغیرها ذخیره کنیم، نوبت به آن رسیده که روی این داده‌ها عملیات انجام دهیم و جریان اجرای برنامه را کنترل کنیم. بدون ساختارهای کنترلی برنامه فقط یک آبشار از بالا به پایین است که از بالا به پایین خوانده و اجرا؛ اما با این ابزارها برنامه هوشمند می‌شود و یکسری از کارها فقط با شروط خاصی اجرا می‌شن.

۱. عملگرهای ریاضی و انتساب یا همان ابزارهای محاسباتی

زبان C عملگرهای استاندارد ریاضی را دارد
عملگرهای اصلی شامل موارد زیر است:

- + جمع
- - تفریق
- * ضرب
- / تقسیم

- **%** باقی‌مانده یا Modulo که باقی‌مانده تقسیم صحیح را برمی‌گرداند.
- نکته بسیار مهم: تقسیم صحیح یا **Truncation** وقتی دو عدد صحیح مانند `int` را بر هم تقسیم می‌کنید، C نتیجه را هم "صحیح" در نظر می‌گیرد و بخش اعشاری را دور می‌ریزد. دقت کنید که گرد نمی‌کند بلکه حذف می‌کند.
- نتیجه $6 / 4$ برابر است با ۱ و نه ۱.۵.
- اگر نتیجه اعشاری می‌خواهید حداقل یکی از اعداد باید اعشاری باشد. مثلاً $6 / 4.0$ برابر است با ۱.۵.
- مثال مقایسه‌ای (دام بزرگ):

```
1. // Common Mistake
2. float avg = 5 / 2;
3. printf("%f", avg);
4. // Output: 2.000000
5. // Reason: Both 5 and 2 are integers, so the result is an integer (2).
```

```
1. // Correct Approach
2. float avg = 5.0 / 2;
3. printf("%f", avg);
4. // Output: 2.500000
5. // Reason: One number (5.0) is a float, so the result is accurate.
```

- عملگرهای انتساب و افزایشی** برنامه‌نویسان C عاشق خلاصه‌نویسی هستند.
- **=** انتساب: مقدار سمت راست را در متغیر سمت چپ می‌ریزد.
 - افزایش `i++`: یعنی یکی به متغیر `i` اضافه کن. این دستور معادل `i = i + 1` است.
 - کاهش `i--`: یعنی یکی از متغیر `i` کم کن.
 - شکل‌های ترکیبی `x += 5`: یعنی `x = x + 5`. این الگو برای بقیه عملگرها مثل `--` و `*` و `/` نیز کار می‌کند.

۲. عملگرهای مقایسه‌ای و منطقی یا همان تصمیم‌گیری

برای اینکه شرط بگذاریم باید بتوانیم مقادیر را مقایسه کنیم. نتیجه این مقایسه‌ها در C عدد ۰ به معنی False یا عدد غیر صفر معمولاً ۱ به معنی True است.

عملگرهای مقایسه‌ای :

- $==$ آیا مساوی است؟
- $!=$ آیا مساوی نیست؟
- $>$ بزرگتر
- $<$ کوچکتر
- $>=$ بزرگتر مساوی
- $<=$ کوچکتر مساوی

دام بزرگ: تفاوت $=$ و $==$ یکی از رایج‌ترین خطاهای برنامه‌نویسان حتی حرفه‌ای‌ها نوشتن $=$ به جای $==$ در شرط‌هاست.

- $\text{if}(x == 3)$ یعنی آیا x برابر با ۳ است؟ درست کار می‌کند.
- $\text{if}(x = 3)$ یعنی مقدار ۳ را در x بریز! این عبارت همیشه "صحیح" ارزیابی می‌شود
- راه حل: همیشه مراقب باشید که $=$ برای مقداردهی است و $==$ برای مقایسه.

عملگرهای منطقی برای ترکیب شرط‌ها :

- $\&\&$ یا همان AND اگر هر دو طرف درست باشند نتیجه درست است.
- $\|\$ یا همان OR اگر حداقل یکی از طرفین درست باشد نتیجه درست است.
- $!$ یا همان NOT نتیجه را برعکس می‌کند. درست را غلط و غلط را درست می‌کند.

۳. ساختارهای شرطی if و switch

حالا که ابزارهای مقایسه را داریم بیایید مسیر برنامه را تغییر دهیم.

دستور if و else ساده‌ترین راه تصمیم‌گیری است. آکولادها {} برای گروه‌بندی دستورات استفاده می‌شوند. (نوشتن else اختیاری هستش و میتونید فقط if بنویسید)

```
1. int score = 15;
2.
3. if (score >= 10) {
4.     printf("Pass\n"); // If condition is true
5. } else {
6.     printf("Fail\n"); // If condition is false
7. }
```

مثال: بررسی سن رای‌دهی (Voting Age Check)

```
1. #include <stdio.h>
2.
3. int main() {
4.     int age = 16;
5.
6.     // Check if age is 18 or older
7.     if (age >= 18) {
8.         printf("You are eligible to vote.\n");
9.     } else {
10.        // Run this if age is less than 18
11.        printf("Sorry, you are too young to vote.\n");
12.    }
13.
14.    return 0;
15. }
```

دستور switch وقتی می‌خواهید یک متغیر را با چندین مقدار ثابت یا Case چک کنید، دستور switch تمیزتر از چندین if پشت سر هم است.

```
1. switch (grade) {
2.     case 'A':
3.         printf("Excellent\n");
4.         break; // Exit the switch block
5.     case 'B':
6.         printf("Good\n");
7.         break;
8.     default: // If no case matches
9.         printf("Unknown Grade\n");
10. }
```

اهمیت: break در C اگر break نگذارید برنامه پس از پیدا کردن مورد صحیح تمام کیس‌های بعدی را هم اجرا می‌کند که به این رفتار Fall-through می‌گویند. پس استفاده از break را فراموش نکنید.

ساختار else if برای شرط‌های تو در تو

گاهی اوقات دنیا سیاه و سفید نیست و ما بیش از دو انتخاب (درست یا غلط) داریم. فرض کنید می‌خواهیم به جای فقط "قبول" یا "رد"، نمره دانشجو را دقیق‌تر بررسی کنیم (مثلاً عالی، خوب، قبول، مردود). در این حالت از else if استفاده می‌کنیم. این ساختار به کامپیوتر می‌گوید: اگر شرط اول درست نبود، این شرط جدید را چک کن، اگر این هم نبود بعدی را چک کن..

مثال: تعیین رتبه دانشجو بر اساس نمره

```

1. #include <stdio.h>
2.
3. int main() {
4.     int score = 17;
5.
6.     // 1. First condition checked: Is score >= 18? (17 >= 18 is False)
7.     if (score >= 18) {
8.         printf("Rank: A (Excellent)\n");
9.     }
10.    // 2. Second condition checked: Is score >= 15? (17 >= 15 is True)
11.    else if (score >= 15) {
12.        printf("Rank: B (Good)\n"); // This block runs.
13.    }
14.    // 3. This condition is skipped because the one above was True.
15.    else if (score >= 10) {
16.        printf("Rank: C (Passed)\n");
17.    }
18.    // 4. This 'else' is also skipped.
19.    else {
20.        printf("Rank: F (Failed)\n");
21.    }
22.
23.    return 0;
24. }
```

تحلیل کد:

برنامه ابتدا شرط اول ($\text{score} \geq 18$) را چک می‌کند. چون ۱۷ بزرگتر از ۱۸ نیست، به سراغ خط ۶ می‌رود.

در خط ۶ شرط ($\text{score} \geq 15$) چک می‌شود. چون ۱۷ بزرگتر از ۱۵ است، این شرط صحیح است.

عبارت Rank: B چاپ می‌شود و برنامه از کل ساختار شرطی خارج می‌شود (دیگر شرط‌های پایین‌تر مثل بزرگتر از ۱۰ را چک نمی‌کند).

نکته مهم: به محض اینکه یکی از شرط‌ها True شود، کامپیوتر آن بلوک را اجرا کرده و بقیه `else if` ها یا `else` را نادیده می‌گیرد.

۴. حلقه‌های تکرار یا Loops

کامپیوترها در انجام کارهای تکراری عالی هستند. حلقه‌ها به ما اجازه می‌دهند یک بلوک کد را بارها اجرا کنیم.

حلقه while تا زمانی که شرط درست باشد تکرار می‌کند. شرط قبل از ورود به حلقه چک می‌شود. (اینجوری که اول شرط چک میشه اگه درست بود اجرا میشه کد و دوباره چک میکنه اگه درست بود دوباره و تا جایی که شرط اشتباه بشه و بعدش از حلقه خارج میشه و ادامه زیر حلقه اجرا میشه)

```
1. int i = 0;
2. while (i < 5) {
3.     printf("Number: %d\n", i);
4.     i++; // Increment counter to avoid infinite loop
5. }
```

حلقه for پرکاربردترین این حلقه تمام اجزای مدیریت تکرار یعنی شروع، شرط و گام حرکت را در یک خط جمع می‌کند. ساختار آن به شکل `for (start; condition; action)` است. (اینجا معمولا میان یه شمارنده میسازن باهش اینجوری که یه متغیر تعریف میکنن و

اول هر دور میان چک میکن اگه شرط درسته که اجرا میشه و شماره یکی اضافه میشه بهش و اگه درست نیست که از حلقه خارج میشه و کد زیر حلقه شروع میکنه به اجرا)

```
1. // Print numbers 0 to 9
2. for (int i = 0; i < 10; i++) {
3.     printf("%d ", i);
4. }
```

نکته: در C رسم است که حلقه‌ها را از ۰ شروع کنیم و از علامت کوچکتر < استفاده کنیم. مثل $i < 10$ برای ۱۰ بار تکرار.

۵. کنترل پیشرفته حلقه با break و continue

گاهی لازم است رفتار استاندارد حلقه را تغییر دهیم:

۱. **break خروج اضطراری:** بلافاصله از کل حلقه یا switch بیرون می‌پرد.

○ مثال: اگر در جستجوی یک عدد بودید و پیدایش کردید دیگر نیازی نیست ادامه دهید و break می‌زنید.

۲. **continue پرش به دور بعد:** بیخیال دستورات باقی‌مانده در این دور از حلقه می‌شود و فوراً به ابتدای دور بعدی می‌رود.

○ مثال: می‌خواهید اعداد ۱ تا ۱۰ را چاپ کنید ولی عدد ۵ را چاپ نکنید. شرط می‌گذارید اگر ۵ بود continue شود.

```
1. for (int i = 0; i < 10; i++) {
2.     if (i == 5) continue; // Skip 5 and go to 6
3.     printf("%d ", i);
4. }
```

مثال: اگه به عدد ۵ رسیدی از حلقه خارج شو

```
1. for (int i = 0; i < 10; i++) {
2.     if (i == 5) {
3.         printf("--- Target reached, breaking out of loop! ---\n");
4.         break;
5.     }
6. }
```

فصل ۳: توابع یا Functions و مدولار سازی

تا اینجای کار تمام کدهایمان را درون تابع main می‌نوشتیم. برای برنامه‌های کوچک این کار مشکلی ندارد اما وقتی برنامه بزرگ می‌شود نوشتن هزاران خط کد داخل یک جا غیرممکن است. راه حل این مشکل شکستن برنامه به قطعات کوچکتر و مستقل به نام تابع است. (اینکار خیلی کمک می‌کند کد تکراری را بنویسیم و خیلی خوبه)

۱. تعریف تابع

تابع یک قطعه کد است که یک کار مشخص را انجام می‌دهد. هر تابع یک نام دارد و می‌تواند ورودی بگیرد و خروجی بدهد. ساختار کلی یک تابع به شکل زیر است

`Return_Type Function_Name (Parameters...)`

۱. `Return_Type` نوع بازگشتی: مشخص می‌کند که تابع چه نوع داده‌ای را به عنوان

نتیجه برمی‌گرداند. مثلاً `int` یا `float`.

۲. `Function_Name` نام تابع: نامی که با آن تابع را صدا می‌زنیم.

۳. `Parameters` پارامترها: متغیرهایی که تابع به عنوان ورودی دریافت می‌کند.

کلمه کلیدی `void` اگر تابعی قرار نیست مقداری را برگرداند و فقط می‌خواهد کاری

انجام دهد مثلاً پیامی را چاپ کند به جای نوع بازگشتی از کلمه کلیدی `void` به معنی پوچ یا خالی استفاده می‌کنیم.

مثال: تابع محاسبه دو برابر این تابع یک عدد می‌گیرد و دو برابر آن را برمی‌گرداند (این تابع رو بالای main تعریف کنید و رگرنه ارور میده بعدا با پروتوتایپ زیر مین رو یاد میگیریم)

```
1. /* Computes double of a number */
2. int Twice(int num) {
3.     int result = num * 2;
4.     return result; // Sends the value back to the caller
5. }
```

نحوه فراخوانی تابع در main برای استفاده از تابع باید نام آن را صدا بزنیم و مقادیر لازم را به آن بدهیم.

```
1. int main() {
2.     int a = 10;
3.     int b = Twice(a); // Calls Twice, passes 10, returns 20
4.     printf("Result: %d\n", b);
5.     return 0;
6. }
```

مثال: محاسبه مساحت مستطیل (Rectangle Area)

```
1. #include <stdio.h>
2.
3. // Function definition: takes length and width, returns area
4. int calculateArea(int length, int width) {
5.     int area = length * width;
6.     return area; // Return the result
7. }
8.
9. int main() {
10.    // Calling the function with arguments 5 and 10
11.    int result = calculateArea(5, 10);
12.
13.    printf("Area of rectangle: %d\n", result);
14.    return 0;
15. }
```

۲. ارسال پارامتر به روش Call by Value

این یکی از مهم‌ترین مفاهیم زبان C است. وقتی شما متغیری را به یک تابع می‌فرستید زبان C خود آن متغیر را نمی‌فرستد بلکه یک کپی از مقدار آن می‌گیرد و به تابع می‌دهد.

نتیجه چیست؟ تغییراتی که درون تابع روی پارامترها انجام می‌شود هیچ تاثیری روی متغیر اصلی در main ندارد. متغیرهای داخل تابع و متغیرهای داخل main در دو دنیای جداگانه زندگی می‌کنند و حافظه مشترکی ندارند.

مثال معروف Swap یا جابجایی: فرض کنید می‌خواهیم تابعی بنویسیم که جای دو عدد را عوض کند. کد زیر کار نمی‌کند:

```
1. void Swap(int x, int y) {
2.     int temp = x;
3.     x = y;
4.     y = temp;
5.     // x and y are swapped here inside the function locally
6. }
7.
8. int main() {
9.     int a = 1;
10.    int b = 2;
11.    Swap(a, b);
12.    // a is still 1 and b is still 2!
13. }
```

در مثال بالا تابع Swap فقط روی کپی‌های x و y کار می‌کند و متغیرهای اصلی a و b دست‌نخورده باقی می‌مانند.
نکته: برای حل این مشکل باید از اشاره‌گرها استفاده کنیم که در فصل‌های آینده یاد می‌گیریم.

تشبیه "کپی جزوه": تصور کنید شما یک جزوه دارید (متغیر در main) دوستان از شما جزوه می‌خواهد. شما از جزوه خود "کپی" می‌گیرید و به او می‌دهید (ارسال به تابع). اگر دوستان روی کپی خودش خط بکشد یا چیزی بنویسد (تغییر مقدار در تابع)، آیا جزوه اصلی شما در کیفیتان تغییر می‌کند؟ خیر.

در زبان C هم وقتی Swap(a, b) را صدا می‌زنید، فقط یک "کپی" از اعداد به تابع فرستاده می‌شود. برای همین متغیرهای اصلی a و b دست‌نخورده باقی می‌مانند.

۳. پروتوتایپ یا Prototype

در زبان C کامپایلر کد را از بالا به پایین می‌خواند. اگر شما تابعی را در `main` صدا بزنید که کد آن پایین‌تر از `main` نوشته شده باشد کامپایلر ارور می‌دهد چون آن تابع را هنوز ندیده است.

راه‌حل: استفاده از پروتوتایپ یا الگوی تابع یعنی معرفی امضای تابع (نام و ورودی/خروجی) قبل از `main` و نوشتن بدنه اصلی تابع بعد از `main`.

```
1. // Prototype: introduces the function to the compiler
2. int Twice(int num);
3.
4. int main() {
5.     printf("%d", Twice(5)); // Compiler knows Twice exists
6.     return 0;
7. }
8.
9. // Function Definition: the actual code
10. int Twice(int num) {
11.     return num * 2;
12. }
```

۴. تقسیم پروژه به چندین فایل (include, Header)

در پروژه‌های واقعی که هزاران خط کد دارند تمام توابع در یک فایل نوشته نمی‌شوند.

- فایل‌های **c**. شامل کدهای اصلی توابع هستند.
- فایل‌های **h**. یا هدر فایل‌ها شامل پروتوتایپ‌ها هستند.

وقتی شما می‌نویسید `#include <stdio.h>` در واقع دارید به کامپایلر می‌گویید: "من کدهای تابع `printf` را ندارم اما پروتوتایپ آن در فایل `stdio.h` موجود است پس قبول کن که این تابع وجود دارد"

مثال عملی: ساخت فایل هدر اختصاصی (Custom Header)

فرض کنید می‌خواهیم یک تابع ساده برای جمع دو عدد داشته باشیم، اما برای جلوگیری از شلوغ شدن کد، آن را در فایل‌های جداگانه مدیریت کنیم. ما پروژه را به سه فایل تقسیم می‌کنیم:

۱. **فایل هدر (calc.h):** در این فایل فقط پروتوتایپ تابع را می‌نویسیم تا به برنامه بگوییم که چنین تابعی وجود دارد.

```
1. // calc.h
2. // Function Prototype: Declares the function without implementing it
3. int add(int a, int b);
```

۲. **فایل سورس (calc.c):** در این فایل بدنه اصلی و دستورات تابع نوشته می‌شود.

```
1. // calc.c
2. #include "calc.h"
3.
4. // Function Definition: The actual logic of the function
5. int add(int a, int b) {
6.     return a + b;
7. }
8.
```

۳. **فایل اصلی برنامه (main.c):** حالا در برنامه اصلی، هدر خودمان را include می‌کنیم تا کامپایلر بداند تابع add معتبر است، حتی اگر کد آن در فایل دیگری باشد.

```
1. // main.c
2. #include <stdio.h> // System header
3. #include "calc.h" // Our custom header
4.
5. int main() {
6.     // Calling the function defined in calc.c
7.     int result = add(10, 20);
8.
9.     printf("Result: %d\n", result);
10.
11.     return 0;
12. }
```

نکته مهم در مورد <> و " ": تفاوت در include

- وقتی می‌نویسیم <stdio.h> کامپایلر در پوشه‌های سیستمی و استاندارد زبان C به دنبال فایل می‌گردد.
- وقتی می‌نویسیم "calc.h" کامپایلر ابتدا در پوشه فعلی پروژه (کنار فایل اصلی) به دنبال فایل می‌گردد.

فصل ۴: آرایه‌ها و رشته‌ها یا Arrays & Strings

تا به اینجا متغیرهای ما فقط می‌توانستند یک مقدار واحد را نگه دارند. مثلاً `int score` فقط یک نمره را ذخیره می‌کرد. اما اگر بخواهیم نمرات ۱۰۰ دانشجو را ذخیره کنیم چه؟ تعریف کردن صد متغیر جداگانه کار عاقلانه‌ای نیست. در اینجاست که آرایه به کمک ما می‌آید.

۱. آرایه‌های یک‌بعدی

آرایه مجموعه‌ای از متغیرهای هم‌نوع است که پشت سر هم در حافظه چیده شده‌اند. تعریف آرایه برای تعریف آرایه باید نوع داده، نام و تعداد خانه‌ها را مشخص کنید. مثال زیر یک آرایه به نام `scores` می‌سازد که می‌تواند ۱۰۰ عدد صحیح را در خود جای دهد.

```
1. int scores[100];  
2. // Sets the first element to 13  
3. scores[0] = 13;  
4. // Sets the last element to 42  
5. scores[99] = 42;
```

قانون شماره‌گذاری از صفر در زبان C شماره‌گذاری خانه‌های آرایه یا همان ایندکس همیشه از ۰ شروع می‌شود. بنابراین اگر آرایه‌ای ۱۰۰ خانه داشته باشد اولین خانه `scores[0]` و آخرین خانه `scores[99]` است.

خطر بزرگ: عدم بررسی مرزها یا **Bounds Checking** زبان C هیچ نظارتی بر شماره خانه‌هایی که استفاده می‌کنید ندارد. اگر شما سعی کنید به خانه `scores[100]` دسترسی پیدا

کنید کامپایلر هیچ خطایی نمی‌دهد اما در زمان اجرا برنامه وارد بخشی از حافظه می‌شود که متعلق به آن نیست. این کار باعث می‌شود برنامه اطلاعات بخش دیگری از حافظه را خراب کند یا به طور ناگهانی بسته شود که به آن Crash می‌گویند. برنامه‌نویس باید خودش مراقب باشد که از مرز آرایه عبور نکند.

```
1. #include <stdio.h>
2.
3. int main() {
4.     // Declare and initialize an array with 4 prices
5.     int prices[4] = {1000, 2500, 500, 1200};
6.
7.     // Loop through index 0 to 3
8.     for (int i = 0; i < 4; i++) {
9.         // Access array element using index i
10.        printf("Price of item %d: %d\n", i, prices[i]);
11.    }
12.
13.    return 0;
14. }
```

۲. آرایه‌های چندبعدی

گاهی اوقات داده‌های ما ساختار جدولی دارند مثل یک صفحه شطرنج یا یک جدول ضرب. در این حالت از آرایه‌های چندبعدی استفاده می‌کنیم. مثال ماتریس برای تعریف یک جدول ۱۰ در ۱۰ کافی است از دو براکت استفاده کنیم.

```
1. int board[10][10];
2. // Set top-left corner
3. board[0][0] = 13;
4. // Set bottom-right corner
5. board[9][9] = 13;
```

در حافظه کامپیوتر این خانه‌ها همچنان به صورت خطی و پشت سر هم ذخیره می‌شوند اما C به ما اجازه می‌دهد به صورت سطر و ستون با آن‌ها کار کنیم.

۳. رشته‌ها در C یا C Strings

در زبان C نوع داده‌ای به نام String وجود ندارد. رشته‌ها در واقع همان آرایه‌ای از کاراکترها char هستند با یک قانون خاص: کاراکتر نال.

کاراکتر پایان رشته یا **Null Terminator** کامپیوتر از کجا می‌فهمد یک جمله تمام شده است؟ در C همیشه بعد از آخرین حرف رشته یک کاراکتر مخصوص با کد عددی صفر قرار می‌گیرد که به آن 0 می‌گویند. این کاراکتر پایان رشته را مشخص می‌کند.

نکته حیاتی: اگر این کاراکتر را فراموش کنید، برنامه نمی‌داند کجا ترمز کند! مثلاً کد زیر خطرناک است: `char name[3] = {'A', 'I', 'i'};` چون جا برای 0 نگذاشتید، ممکن است در خروجی بعد از Ali کاراکترهای عجیب و غریب حافظه چاپ شود. همیشه طول آرایه را یک واحد بیشتر از تعداد حروف در نظر بگیرید

```
1. char localString[10];
2. // Copies "binky" into the array
3. strcpy(localString, "binky");
```

در مثال بالا کلمه `binky` پنج حرف دارد اما در حافظه ۶ خانه اشغال می‌کند زیرا حرف ششم همان `\0` است.

تفاوت 'A' و "A"

- 'A' یک کاراکتر تک است و در واقع یک عدد صحیح کوچک است که کد اسکی آن حرف را نگه می‌دارد.
- "A" یک رشته است که شامل دو خانه حافظه می‌شود: یکی برای خود حرف A و دیگری برای `\0` پایان رشته.

۴. توابع کتابخانه‌ای رشته در `string.h`

چون رشته‌ها آرایه هستند نمی‌توانید با علامت مساوی `=` آن‌ها را کپی کنید یا با `==` آن‌ها را مقایسه کنید. برای این کارها باید از کتابخانه استاندارد `<string.h>` استفاده کنید. تابع `strcpy` برای کپی کردن این تابع محتوای یک رشته را درون رشته دیگر کپی می‌کند. فرمت آن به شکل `strcpy(destination, source)` است.

```
1. char s1[20];
2. strcpy(s1, "Hello"); // Correct way to assign string
```

تابع `strlen` برای طول رشته این تابع تعداد کاراکترهای رشته را برمی‌گرداند اما کاراکتر نال `\0` را نمی‌شمارد.

تابع `strcmp` برای مقایسه برای اینکه بفهمیم دو رشته یکسان هستند یا نه از این تابع استفاده می‌کنیم.

- اگر دو رشته مساوی باشند خروجی تابع ۰ است.
- اگر مساوی نباشند عددی غیر از صفر برمی‌گرداند.

نکته مهم: هرگز برای مقایسه رشته‌ها از `==` استفاده نکنید چون این کار آدرس حافظه آن‌ها را مقایسه می‌کند نه محتوای حروف را.

فصل ۵: اشاره گر ها یا Pointers و

استراکچر یا Structs

به بخش حرفه‌ای زبان C خوش آمدید. اشاره گر ها ابزاری هستند که به شما قدرت کنترل مستقیم حافظه را می‌دهند. اگر متغیرها را مثل جعبه‌هایی تصور کنیم که داده‌ها درون آنها قرار می‌گیرند، اشاره گر ها مثل آدرس پستی آن جعبه‌ها هستند.

۱. اشاره گر چیست؟

اشاره گر یا پوینتر متغیری است که مقدارش یک عدد معمولی نیست، بلکه آدرس یک متغیر دیگر است. به متغیری که به آن اشاره می‌شود Pointe یا "اشاره شونده" می‌گویند. برای درک بهتر، یک صندوق را تصور کنید. متغیر معمولی (x) خود صندوق است که داخلش مثلاً یک عدد ۱۰ قرار دارد. اما اشاره گر (p) مثل یک تکه کاغذ کوچک است که ما روی آن "شماره پلاک" یا آدرس آن صندوق را نوشته‌ایم، نه خود عدد ۱۰ را. وقتی می‌گوییم *p یعنی برو به پلاکی که روی کاغذ نوشته شده و محتوای صندوق را بخوان

نحوه تعریف برای تعریف یک اشاره گر از علامت ستاره * استفاده می‌کنیم. مثلاً `char*` نوعی اشاره گر است که به یک کاراکتر اشاره می‌کند.

```
1. int* p; // p is a pointer to an integer
```

عملگرهای حیاتی & و * برای کار با اشاره‌گرها دو عملگر مهم داریم:

۱. عملگر & یا آدرس‌گیر: این عملگر آدرس متغیر را به ما می‌دهد. وقتی

می‌نویسیم &i یعنی "آدرس خانه حافظه متغیر i را به من بده."

۲. عملگر * یا دسترسی به محتوا: اگر یک اشاره‌گر داشته باشیم و بخواهیم ببینیم

داخل آن آدرس چه چیزی نوشته شده است، از این عملگر استفاده می‌کنیم که به

آن Dereference می‌گویند.

```
1. int i = 13;
2. int* p;
3.
4. p = &i; // Set p to point to i (p holds the address of i)
5. *p = 42; // Change the value at that address to 42
6. // Now i is 42
```

در کد بالا با اینکه ما مستقیماً مقدار i را تغییر ندادیم اما چون p به i اشاره می‌کرد، تغییر p باعث تغییر i شد.

۲. استراکچر یا Struct

در فصل‌های قبل فقط می‌توانستیم یک عدد یا یک حرف را ذخیره کنیم. اما اگر بخواهیم مشخصات یک "دانشجو" شامل نام و شماره دانشجویی را یکجا داشته باشیم چه؟ استراکچر به ما اجازه می‌دهد متغیرهای مختلف را در یک بسته واحد گروه‌بندی کنیم.

تعریف و استفاده برای ساخت نوع داده جدید از کلمه کلیدی struct استفاده می‌کنیم. فراموش نکنید که در پایان تعریف علامت ; لازم است.

```
1. struct Student {
2.     int id;
3.     int age;
4. }; // Semicolon is required!
```

دسترسی با نقطه . برای دسترسی به اجزای داخلی یک استراکچر از عملگر نقطه استفاده می کنیم.

```
1. struct Student s1;
2. s1.id = 99123;
3. s1.age = 20;
```

۳. ارسال با مرجع یا Call by Reference

در فصل ۳ گفتیم که C کپی متغیرها را به تابع می فرستد و تابع نمی تواند متغیر اصلی را تغییر دهد. اما حالا با اشاره گر ها راه حل را داریم: به جای فرستادن خود متغیر، آدرس آن را می فرستیم.

تفاوت این دو روش را این طور تصور کنید: ۱. روش قبلی (Value) مثل این بود که یک فایل Word را کپی کنید و برای کسی ایمیل کنید. اگر او فایل را تغییر دهد، فایل اصلی در کامپیوتر شما هیچ تغییری نمی کند. ۲. روش فعلی (Reference) مثل این است که فایل را در فضای ابری (Cloud) بگذارید و فقط "لینک" آن را به دوستان بدهید. حالا هر تغییری که او بدهد، روی فایل اصلی شما هم اعمال می شود چون هر دو دارید روی یک آدرس کار می کنید

تابع Swap یا جابجایی این مثال کلاسیک نشان می دهد چطور دو متغیر را در main با استفاده از یک تابع دیگر جابجا کنیم.

۱. در تعریف تابع ورودی ها را به صورت اشاره گر `int*` می نویسیم.
۲. درون تابع برای دسترسی به مقادیر از `*` استفاده می کنیم.
۳. هنگام صدا زدن تابع، آدرس متغیرها را با `&` می فرستیم.

```
1. // Function receives addresses (pointers)
2. void Swap(int* a, int* b) {
3.     int temp = *a; // Get value at address a
4.     *a = *b;       // Put value at b into address a
5.     *b = temp;     // Put temp into address b
```

```

6. }
7.
8. int main() {
9.     int x = 10;
10.    int y = 20;
11.    Swap(&x, &y); // Pass addresses of x and y
12.    // Now x is 20 and y is 10
13.    return 0;
14. }

```

در اینجا چون تابع Swap آدرس‌های حافظه x و y را داشت، توانست مستقیماً محتوای آنها را تغییر دهد.

۴. اشاره‌گر به استراکچر

بسیار پیش می‌آید که ما یک اشاره‌گر داشته باشیم که به یک استراکچر اشاره می‌کند. عملگر فلش یا >- فرض کنید stPtr یک اشاره‌گر به استراکچر دانشجو است. برای دسترسی به فیلد id می‌توانیم بنویسیم (*stPtr).id اما این نگارش سخت و ناخوانا است. زبان C یک راه میانبر زیبا ارائه کرده است: عملگر فلش >- که در سمت راست اشاره‌گر می‌آید.

```

1. struct Student s1;
2. s1.id = 100;
3.
4. struct Student* ptr;
5. ptr = &s1; // ptr now points to s1
6.
7. // Standard way (hard to read)
8. (*ptr).id = 101;
9.
10. // Arrow operator way (preferred)
11. ptr->id = 102; // Change s1.id to 102

```

هر دو خط کد بالا دقیقاً یک کار را انجام می‌دهند اما استفاده از >- بسیار رایج‌تر و خواناتر است.

فصل ۶: دریافت ورودی و تبدیل انواع داده

تا به اینجا، تمام داده‌های برنامه‌های ما "ثابت (Hardcoded)" بودند؛ یعنی مقادیر متغیرها را خودمان در کد می‌نوشتیم مثل `int a = 10` اما برنامه‌های واقعی نیاز دارند با کاربر تعامل داشته باشند و اطلاعات را از او بگیرند. همچنین گاهی لازم است نوع داده‌ها را تغییر دهیم (مثلاً متن را به عدد تبدیل کنیم) تا محاسبات دقیق‌تری داشته باشیم. در این فصل، با تابع قدرتمند `scanf`، مفهوم `Type Casting` و توابع تبدیل رشته آشنا می‌شویم.

۱. دریافت ورودی با `scanf`

در زبان C، ابزار اصلی ما برای خواندن ورودی از کیبورد، تابع `scanf` است که در کتابخانه `<stdio.h>` قرار دارد. نام این تابع مخفف **Scan Formatted** است.

ساختار دستور

این تابع منتظر می‌ماند تا کاربر تایپ کند و کلید `Enter` را بزند.

```
1. scanf("Format Specifier", &Variable);
```

قانون حیاتی: علامت `&` (Address-of Operator)

نکته‌ای که بسیاری از مبتدیان فراموش می‌کنند و باعث بسته شدن ناگهانی برنامه (`Crash`) می‌شود، علامت `&` است.

- وقتی از `printf` استفاده می‌کنیم، فقط می‌خواهیم مقدار متغیر را بخوانیم.

- اما وقتی از `scanf` استفاده می‌کنیم، می‌خواهیم مقدار جدیدی را در حافظه بنویسیم. پس باید آدرس خانه حافظه را به تابع بدهیم تا بداند داده را کجا ذخیره کند.

جدول فرمت‌های رایج (Format Specifiers)

باید دقیقاً به `scanf` بگوییم منتظر چه نوع داده‌ای باشد:

نوع داده	فرمت	مثال کد
عدد صحیح (int)	%d	<code>scanf("%d", &age);</code>
عدد اعشاری (float)	%f	<code>scanf("%f", &weight);</code>
عدد اعشاری دقیق (double)	%lf	<code>scanf("%lf", &pi);</code>
کاراکتر (char)	%c	<code>scanf(" %c", &grade);</code>
رشته/متن (String)	%s	<code>scanf("%s", name);</code>

۲. مثال‌های کاربردی دریافت ورودی

مثال ۱: دریافت یک عدد صحیح (ساده)

در این برنامه سن کاربر را می‌پرسیم و به او نمایش می‌دهیم.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int age;
5.
6.     // Prompt user for input
7.     printf("Please enter your age: ");
8.
9.     // Read integer from keyboard. Notice the '&' before variable name.
10.    scanf("%d", &age);
11.
12.    // Display the result
13.    printf("You are %d years old.\n", age);

```

```
14.  
15.     return 0;  
16. }
```

مثال ۲: دریافت چند ورودی همزمان

شما می‌توانید چند عدد را جداگانه یا یکجا دریافت کنید.

```
1. #include <stdio.h>  
2.  
3. int main() {  
4.     int length, width;  
5.  
6.     // Ask for two numbers  
7.     printf("Enter length and width (separated by space): ");  
8.  
9.     // Taking two inputs at once. User types: "10 20" then Enter  
10.    scanf("%d %d", &length, &width);  
11.  
12.    int area = length * width;  
13.  
14.    printf("Area of rectangle is: %d\n", area);  
15.  
16.    return 0;  
17. }
```

مثال ۳: کار با اعداد اعشاری (double)

نکته مهم: برای double در printf از %f استفاده می‌شود، اما در scanf حتماً باید از %lf (Long Float) استفاده کنید.

```
1. #include <stdio.h>  
2.  
3. int main() {  
4.     double price;  
5.  
6.     printf("Enter the price: ");  
7.  
8.     // Reading a double precision number requires %lf  
9.     scanf("%lf", &price);  
10.  
11.    // Calculate tax (9%)  
12.    double total = price * 1.09;  
13.  
14.    // %.2f limits output to 2 decimal places  
15.    printf("Total Price with tax: %.2f\n", total);  
16.  
17.    return 0;  
18. }
```

۳. استثناهای رشته‌ها (String)

نام آرایه (رشته) در زبان C خودش یک اشاره‌گر به آدرس است. بنابراین برای رشته‌ها نیازی به علامت & نیست.

```
1. char name[50];
2. printf("Enter your name: ");
3. scanf("%s", name); // No '&' needed here
```

توجه *scanf* با %s فقط تا اولین فاصله (Space) را می‌خواند

۴. تبدیل انواع داده (Type Casting)

گاهی اوقات لازم است نوع یک متغیر را به نوع دیگری تبدیل کنیم.

الف) تبدیل خودکار (Implicit)

وقتی یک داده کوچک‌تر را در ظرف بزرگ‌تر می‌ریزید، کامپایلر خودکار این کار را انجام می‌دهد.

```
1. int num = 10;
2. float f = num; // Automatically becomes 10.0
```

ب) تبدیل دستی (Explicit Casting)

این حالت مهم‌تر است. ساختار آن به این صورت است که نوع مقصد را داخل پرانتز قبل از مقدار می‌نویسیم. value (type) :

۱. مشکل تقسیم صحیح (The Integer Division Trap)

در C، تقسیم دو عدد صحیح (int / int) همیشه نتیجه صحیح می‌دهد و اعشار حذف می‌شود.

```
1. int a = 5;
2. int b = 2;
3. float res = a / b; // Result is 2.0 (Incorrect!)
```

چون 5 تقسیم بر 2 می‌شود 2.5 بخش اعشاری دور ریخته می‌شود.

راه حل با کستینگ:

باید حداقل یکی از اعداد را موقتاً به float تبدیل کنیم.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int sum = 17;
5.     int count = 5;
6.
7.     // WRONG: Integer division happens first
8.     float avg1 = sum / count;
9.
10.    // CORRECT: Explicitly cast 'sum' to float before division
11.    float avg2 = (float)sum / count;
12.
13.    printf("Wrong Average: %.2f\n", avg1); // Output: 3.00
14.    printf("Correct Average: %.2f\n", avg2); // Output: 3.40
15.
16.    return 0;
17. }

```

۲. حذف اعشار (Float to Int)

اگر بخواهید فقط بخش صحیح یک عدد اعشاری را نگه دارید، می‌توانید آن را به `int` کست کنید. (دقت کنید: عدد گرد نمی‌شود، بلکه بریده می‌شود).

```

1. float pi = 3.99;
2. int intPi = (int)pi; // Becomes 3

```

۵. تبدیل رشته به عدد (String to Number)

گاهی اوقات ما یک عدد را به صورت متنی (داخل دابل کوتیشن) داریم، مثلاً "123". این حالت در خواندن از فایل یا ورودی‌های خاص بسیار پیش می‌آید. برای انجام محاسبات روی این متون، باید آن‌ها را تبدیل کنیم.

برای این کار از کتابخانه `<stdlib.h>` و توابع خانواده `ato` استفاده می‌کنیم.

توابع تبدیل:

نام تابع	مخفف	کاربرد	مثال
<code>atoi</code>	ASCII to Integer	تبدیل رشته به <code>int</code>	"25" → 25
<code>atof</code>	ASCII to Float	تبدیل رشته به <code>double</code>	"3.14" → 3.14

مثال عملی: ماشین حساب متنی

```
1. #include <stdio.h>
2. #include <stdlib.h> // Required for atoi and atof
3.
4. int main() {
5.     // Examples of numbers stored as text strings
6.     char strInt[] = "100";
7.     char strFloat[] = "45.50";
8.
9.     // 1. Convert String to Integer using atoi
10.    int num1 = atoi(strInt);
11.
12.    // 2. Convert String to Float/Double using atof
13.    double num2 = atof(strFloat);
14.
15.    // Now we can perform math operations
16.    double sum = num1 + num2;
17.
18.    printf("String 1: %s\n", strInt);
19.    printf("String 2: %s\n", strFloat);
20.    printf("Sum Result: %.2f\n", sum); // Output: 145.50
21.
22.    return 0;
23. }
```

نکته: اگر رشته‌ای که عدد نیست مثل "Hello" را به این توابع بدهید، معمولاً خروجی 0 خواهد بود.

فصل ۷: ۲۰ مثال کاربردی و کوتاه

در این فصل تمام مفاهیم قبلی را در قالب کدهای کوچک و قابل اجرا مرور می‌کنیم. توجه داشته باشید که کدها بجز توابع در بدنه تابع `main` باید نوشته بشن

۱. سلام دنیا

ساده‌ترین برنامه برای اطمینان از نصب صحیح کامپایلر.

```
1. #include <stdio.h>
2.
3. int main() {
4.     printf("Hello World\n");
5.     return 0;
6. }
```

۲. دریافت عدد از کاربر

خواندن یک عدد صحیح از ورودی و چاپ آن.

```
1. int num;
2. printf("Enter a number: ");
3. scanf("%d", &num); // Read integer from user
4. printf("You entered: %d", num);
```

۳. جمع دو عدد صحیح

گرفتن دو عدد و نمایش حاصل جمع آنها.

```
1. int a = 5, b = 10;
2. int sum = a + b;
3. printf("Sum is: %d", sum); // Output: 15
```

۴. بررسی زوج یا فرد بودن

استفاده از عملگر باقی مانده برای تشخیص زوجیت.

```
1. int n = 7;
2. if (n % 2 == 0) {
3.     printf("Even");
4. } else {
5.     printf("Odd"); // Output: Odd
6. }
```

۵. تبدیل دما از سلسیوس به فارنهایت

فرمول ریاضی ساده با استفاده از اعداد اعشاری.

```
1. float c = 25.0;
2. float f = (c * 9 / 5) + 32;
3. printf("Temp in Fahrenheit: %.2f", f);
```

۶. پیدا کردن بزرگترین عدد بین سه متغیر

استفاده از ساختار شرطی تو در تو.

```
1. int x = 10, y = 20, z = 15;
2. if (x > y && x > z) printf("%d is max", x);
3. else if (y > x && y > z) printf("%d is max", y); // This runs
4. else printf("%d is max", z);
```

۷. چاپ ایام هفته با Switch

تبدیل عدد ورودی به نام روز هفته.

```
1. int day = 3;
2. switch(day) {
3.     case 1: printf("Saturday"); break;
4.     case 2: printf("Sunday"); break;
5.     case 3: printf("Monday"); break; // Output: Monday
6.     default: printf("Invalid day");
7. }
```

۸. چاپ اعداد ۱ تا ۱۰

استفاده از حلقه for برای شمارش.

```
1. for (int i = 1; i <= 10; i++) {
2.     printf("%d ", i); // Prints 1 2 3 ... 10
3. }
```

۹. محاسبه فاکتوریل

استفاده از حلقه while برای ضرب متوالی اعداد.

```
1. int n = 5, fact = 1;
2. while (n > 0) {
3.     fact *= n; // fact = fact * n
4.     n--;
5. }
6. printf("Factorial: %d", fact); // Output: 120
```

۱۰. جدول ضرب ۵

چاپ مضرب‌های عدد ۵ با حلقه.

```
1. for (int i = 1; i <= 10; i++) {
2.     printf("5 * %d = %d\n", i, 5 * i);
3. }
```

۱۱. جمع عناصر آرایه

پیمایش یک آرایه و جمع زدن تمام خانه‌ها.

```
1. int arr[] = {1, 2, 3, 4, 5};
2. int sum = 0;
3. for (int i = 0; i < 5; i++) {
4.     sum += arr[i];
5. }
6. printf("Sum: %d", sum); // Output: 15
```

۱۲. پیدا کردن ماکسیمم در آرایه

جستجو برای بزرگترین عدد در یک لیست.

```
1. int arr[] = {12, 45, 2, 99, 4};
2. int max = arr[0];
3. for (int i = 1; i < 5; i++) {
4.     if (arr[i] > max) max = arr[i];
5. }
6. printf("Max is: %d", max); // Output: 99
7.
```

۱۳. شمارش حروف یک رشته

محاسبه طول رشته بدون استفاده از تابع `strlen`.

```
1. char str[] = "Hello";
2. int len = 0;
3. while (str[len] != '\0') {
4.     len++;
5. }
6. printf("Length: %d", len); // Output: 5
```

۱۴. کپی کردن رشته

استفاده از تابع `strcpy` برای مقداردهی به رشته.

```
1. char src[] = "Copy Me";
2. char dest[20];
3. strcpy(dest, src); // Copy content of src to dest
4. printf("Dest: %s", dest);
```

۱۵. تابع محاسبه توان

نوشتن یک تابع جداگانه که عدد را به توان می‌رساند.

```
1. int Power(int base, int exp) {
2.     int res = 1;
3.     for (int i = 0; i < exp; i++) res *= base;
4.     return res;
5. }
6. // Usage in main: int p = Power(2, 3); // 8
```

۱۶. کار با اشاره گر ساده

تغییر مقدار متغیر از طریق آدرس آن.

```
1. int a = 10;
2. int *p = &a; // p holds address of a
3. *p = 20;      // Change value of a through pointer
4. printf("a is now: %d", a); // Output: 20
```

۱۷. جابجایی دو عدد با اشاره گر یا Swap

تابعی که جای دو متغیر را در حافظه عوض می‌کند.

```
1. void Swap(int *x, int *y) {
2.     int temp = *x;
3.     *x = *y;
4.     *y = temp;
5. }
6. // Usage: Swap(&a, &b);
```

۱۸. تعریف استراکچر دانشجو

ذخیره نام و معدل یک دانشجو در یک بسته.

```
1. struct Student {
2.     char name[50];
3.     float gpa;
4. };
5. struct Student s1 = {"Ali", 18.5};
6. printf("%s : %.2f", s1.name, s1.gpa);
```

۱۹. چاپ کد اسکی کاراکتر

نمایش کد عددی متناظر با یک حرف.

```
1. char ch = 'A';
2. printf("ASCII code of %c is %d", ch, ch); // Output: 65
```

۲۰. ماشین حساب ساده

ترکیب switch و ورودی کاربر برای چهار عمل اصلی.

```
1. char op;
2. int a, b;
3. printf("Enter a op b: "); // Like: 2 + 3
4. scanf("%d %c %d", &a, &op, &b);
5. switch(op) {
6.     case '+': printf("%d", a + b); break;
7.     case '-': printf("%d", a - b); break;
8.     case '*': printf("%d", a * b); break;
9.     case '/': printf("%d", a / b); break;
10. }
```

فصل ۸: ۲۰ تمرین

در این فصل بیست مسئله معروف را که در اکثر امتحانات و کلاس‌های برنامه‌نویسی مطرح می‌شوند بررسی می‌کنیم.

۱. چاپ هرم ستاره‌ای

این یکی از معروف‌ترین تمرین‌ها برای یادگیری حلقه‌های تو در تو است. ما نیاز به یک حلقه برای سطرها و دو حلقه داخلی برای چاپ فاصله‌ها و ستاره‌ها داریم.

```
1. #include <stdio.h>
2.
3. int main() {
4.     int rows = 5;
5.     // Outer loop for number of rows
6.     for (int i = 1; i <= rows; i++) {
7.         // Loop for printing spaces
8.         for (int j = 1; j <= rows - i; j++) {
9.             printf(" ");
10.        }
11.        // Loop for printing stars
12.        for (int k = 1; k <= (2 * i - 1); k++) {
13.            printf("*");
14.        }
15.        printf("\n"); // Move to next line
16.    }
17.    return 0;
18. }
```

خروجی برنامه:

```
1.      *
2.     ***
3.    *****
4.   *
5.  *****
```

۲. مثلث اعداد

چاپ مثلثی که در آن اعداد پشت سر هم تکرار می‌شوند. این تمرین برای درک رابطه بین شمارنده حلقه‌ها عالی است.

```
1. #include <stdio.h>
2.
3. int main() {
4.     int rows = 5;
5.     for (int i = 1; i <= rows; i++) {
6.         for (int j = 1; j <= i; j++) {
7.             printf("%d ", j); // Print number
8.         }
9.         printf("\n");
10.    }
11.    return 0;
12. }
```

خروجی برنامه:

```
1. 1
2. 1 2
3. 1 2 3
4. 1 2 3 4
5. 1 2 3 4 5
```

۳. جدول ضرب

تولید جدول ضرب کلاسیک ده در ده با استفاده از دو حلقه تو در تو و فرمت‌بندی خروجی برای مرتب بودن ستون‌ها.

```
1. #include <stdio.h>
2.
3. int main() {
4.     for (int i = 1; i <= 10; i++) {
5.         for (int j = 1; j <= 10; j++) {
6.             printf("%4d", i * j); // %4d adds padding for alignment
7.         }
8.         printf("\n");
9.     }
10.    return 0;
11. }
```

خروجی برنامه:

```
1. 1 2 3 ... 10
2. 2 4 6 ... 20
```

```

3.    ...
4.    10  20  30 ... 100

```

۴. تشخیص عدد اول

عدد اول عددی است که فقط بر خودش و یک بخش پذیر باشد. برای بررسی آن یک حلقه از دو تا نصف آن عدد ایجاد می کنیم و اگر بخش پذیری دیدیم یعنی اول نیست.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int n = 17, isPrime = 1; // 1 means true
5.
6.     if (n < 2) isPrime = 0;
7.
8.     for (int i = 2; i <= n / 2; i++) {
9.         if (n % i == 0) {
10.            isPrime = 0; // Found a divisor, not prime
11.            break;
12.        }
13.    }
14.
15.    if (isPrime) printf("Prime Number");
16.    else printf("Not Prime");
17.    return 0;
18. }

```

خروجی برنامه:

```

1. Prime Number

```

۵. سری فیبوناچی

سری فیبوناچی دنباله ای است که هر عدد جمع دو عدد قبلی خودش است و با صفر و یک شروع می شود.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int n = 10, t1 = 0, t2 = 1, nextTerm;
5.     printf("Fibonacci Series: %d, %d, ", t1, t2);
6.
7.     for (int i = 3; i <= n; i++) {
8.         nextTerm = t1 + t2;
9.         printf("%d, ", nextTerm);
10.        t1 = t2;
11.        t2 = nextTerm;

```

```
12.     }
13.     return 0;
14. }
```

خروجی برنامه:

```
1. Fibonacci Series: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34,
```

۶. مقلوب یا برعکس کردن یک عدد

این الگوریتم ارقام یک عدد را از انتها جدا کرده و یک عدد جدید می‌سازد. مثلاً ۱۲۳ تبدیل می‌شود به ۳۲۱.

```
1. #include <stdio.h>
2.
3. int main() {
4.     int n = 12345, reverse = 0, remainder;
5.
6.     while (n != 0) {
7.         remainder = n % 10; // Get last digit
8.         reverse = reverse * 10 + remainder;
9.         n /= 10; // Remove last digit
10.    }
11.    printf("Reversed number = %d", reverse);
12.    return 0;
13. }
```

خروجی برنامه:

```
1. Reversed number = 54321
```

۷. ادغام دو آرایه یا Merge Arrays

یک تمرین بسیار کاربردی برای کار با آرایه‌ها. در اینجا دو آرایه جداگانه داریم و می‌خواهیم آن‌ها را پشت سر هم در یک آرایه سوم ذخیره کنیم.

```
1. #include <stdio.h>
2.
3. int main() {
4.     int arr1[] = {1, 2, 3};
5.     int arr2[] = {4, 5, 6};
6.     int merged[6]; // Size must be sum of both
7.
8.     // Copy first array
9.     for (int i = 0; i < 3; i++) {
10.        merged[i] = arr1[i];
11.    }
```

```

12.
13.     // Copy second array
14.     for (int i = 0; i < 3; i++) {
15.         merged[i + 3] = arr2[i]; // Shift index by size of first array
16.     }
17.
18.     // Print result
19.     for (int i = 0; i < 6; i++) printf("%d ", merged[i]);
20.     return 0;
21. }

```

خروجی برنامه:

```
1. 1 2 3 4 5 6
```

۸. محاسبه فاکتوریل با بازگشت

توابع بازگشتی توابعی هستند که خودشان را صدا می‌زنند. محاسبه فاکتوریل ساده‌ترین مثال برای درک این مفهوم است.

```

1. #include <stdio.h>
2.
3. int factorial(int n) {
4.     if (n == 0) return 1; // Base case
5.     else return n * factorial(n - 1); // Recursive call
6. }
7.
8. int main() {
9.     int num = 5;
10.    printf("Factorial of %d is %d", num, factorial(num));
11.    return 0;
12. }

```

خروجی برنامه:

```
1. Factorial of 5 is 120
```

۹. حذف عناصر تکراری از آرایه

در این مسئله یک آرایه داریم که ممکن است اعداد تکراری داشته باشد. می‌خواهیم فقط عناصر منحصر به فرد یا یونیک را چاپ کنیم.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {1, 2, 2, 3, 4, 4, 5};
5.     int n = 7;

```

```

6.
7.     printf("Unique elements: ");
8.     for (int i = 0; i < n; i++) {
9.         int isDuplicate = 0;
10.        // Check if element appeared before
11.        for (int j = 0; j < i; j++) {
12.            if (arr[i] == arr[j]) {
13.                isDuplicate = 1;
14.                break;
15.            }
16.        }
17.        if (!isDuplicate) printf("%d ", arr[i]);
18.    }
19.    return 0;
20. }

```

خروجی برنامه:

```
1. Unique elements: 1 2 3 4 5
```

۱۰. بزرگترین مقسوم علیه مشترک یا BMM

برای پیدا کردن ب.م.م دو عدد از الگوریتم اقلیدس استفاده می‌کنیم که مبتنی بر تکرار تقسیم یا تفریق است.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int n1 = 50, n2 = 60;
5.     while(n1 != n2) {
6.         if(n1 > n2) n1 -= n2;
7.         else n2 -= n1;
8.     }
9.     printf("GCD = %d", n1);
10.    return 0;
11. }

```

خروجی برنامه:

```
1. GCD = 10
```

۱۱. جستجوی خطی یا Linear Search

ساده‌ترین روش جستجو که در آن تک تک خانه‌های آرایه را چک می‌کنیم تا عدد مورد نظر را پیدا کنیم.

```
1. #include <stdio.h>
```

```

2.
3. int main() {
4.     int arr[] = {2, 4, 0, 1, 9};
5.     int target = 1, found = 0;
6.
7.     for (int i = 0; i < 5; i++) {
8.         if (arr[i] == target) {
9.             printf("Element found at index %d", i);
10.            found = 1;
11.            break;
12.        }
13.    }
14.    if (!found) printf("Element not found");
15.    return 0;
16. }

```

خروجی برنامه:

1. Element found at index 3

۱۲. جستجوی دودویی یا Binary Search

این روش فقط روی آرایه‌های مرتب کار می‌کند. آرایه را نصف می‌کنیم و می‌بینیم عدد ما در کدام نیمه است. این روش بسیار سریع‌تر از جستجوی خطی است.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {10, 20, 30, 40, 50}; // Must be sorted
5.     int target = 40;
6.     int left = 0, right = 4, mid;
7.
8.     while (left <= right) {
9.         mid = left + (right - left) / 2;
10.        if (arr[mid] == target) {
11.            printf("Found at index %d", mid);
12.            break;
13.        }
14.        if (arr[mid] < target) left = mid + 1;
15.        else right = mid - 1;
16.    }
17.    return 0;
18. }

```

خروجی برنامه:

1. Found at index 3

۱۳. مرتب‌سازی حبابی Bubble Sort

معروف‌ترین الگوریتم مرتب‌سازی برای آموزش که در آن عناصر دو به دو مقایسه و جابجا می‌شوند تا بزرگترین عنصر مثل حباب به انتهای آرایه برود.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {64, 34, 25, 12, 22};
5.     int n = 5, temp;
6.
7.     for (int i = 0; i < n - 1; i++) {
8.         for (int j = 0; j < n - i - 1; j++) {
9.             if (arr[j] > arr[j + 1]) {
10.                // Swap logic
11.                temp = arr[j];
12.                arr[j] = arr[j + 1];
13.                arr[j + 1] = temp;
14.            }
15.        }
16.    }
17.
18.    printf("Sorted array: ");
19.    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
20.    return 0;
21. }
```

خروجی برنامه:

```
1. Sorted array: 12 22 25 34 64
```

۱۴. جمع دو ماتریس

جمع کردن خانه‌های متناظر دو آرایه دو بعدی و ریختن نتیجه در ماتریس سوم.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int a[2][2] = {{1, 2}, {3, 4}};
5.     int b[2][2] = {{5, 6}, {7, 8}};
6.     int sum[2][2];
7.
8.     for (int i = 0; i < 2; i++) {
9.         for (int j = 0; j < 2; j++) {
10.            sum[i][j] = a[i][j] + b[i][j];
11.            printf("%d ", sum[i][j]);
12.        }
13.        printf("\n");
14.    }
```

```

14.     }
15.     return 0;
16. }

```

خروجی برنامه:

```

1. 6 8
2. 10 12

```

۱۵. محاسبه میانگین عناصر آرایه

یک تمرین بسیار رایج آماری که در آن تمام عناصر آرایه را جمع می‌زنیم و بر تعداد تقسیم می‌کنیم. نکته مهم تبدیل نوع داده به float برای دقت اعشاری است.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {10, 20, 30, 40, 50};
5.     int n = 5;
6.     float sum = 0.0, avg;
7.
8.     for (int i = 0; i < n; i++) {
9.         sum += arr[i];
10.    }
11.
12.    avg = sum / n;
13.    printf("Average = %.2f", avg);
14.    return 0;
15. }

```

خروجی برنامه:

```

1. Average = 30.00

```

۱۶. شمارش تعداد حروف صدا دار

یک رشته را پیمایش می‌کنیم و با استفاده از دستور switch یا if بررسی می‌کنیم چند حرف صدا دار مثل a یا e در آن وجود دارد.

```

1. #include <stdio.h>
2.
3. int main() {
4.     char s[] = "University";
5.     int count = 0;
6.     for (int i = 0; s[i] != '\0'; i++) {
7.         char ch = s[i];
8.         if (ch=='a' || ch=='e' || ch=='i' || ch=='o' || ch=='u' ||

```

```

9.             ch=='A' || ch=='E' || ch=='I' || ch=='O' || ch=='U') {
10.             count++;
11.         }
12.     }
13.     printf("Vowels count: %d", count);
14.     return 0;
15. }

```

خروجی برنامه:

1. Vowels count: 4

۱۷. چرخش آرایه یا Array Rotation

در این الگوریتم عناصر آرایه را یک واحد به سمت چپ شیفت می‌دهیم. عنصر اول به انتهای آرایه می‌رود و بقیه یکی به چپ می‌آیند.

C

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {1, 2, 3, 4, 5};
5.     int first = arr[0]; // Store first element
6.     int n = 5;
7.
8.     // Shift elements left
9.     for (int i = 0; i < n - 1; i++) {
10.        arr[i] = arr[i + 1];
11.    }
12.    arr[n - 1] = first; // Place first element at end
13.
14.    for(int i = 0; i < n; i++) printf("%d ", arr[i]);
15.    return 0;
16. }

```

خروجی برنامه:

1. 2 3 4 5 1

۱۸. تبدیل حروف کوچک به بزرگ

در جدول اسکی حروف کوچک با حروف بزرگ ۳۲ واحد فاصله دارند. با کم کردن عدد ۳۲ از کاراکتر کوچک به کاراکتر بزرگ می‌رسیم.

```

1. #include <stdio.h>
2.

```

```

3. int main() {
4.     char str[] = "hello";
5.     for (int i = 0; str[i] != '\0'; i++) {
6.         if(str[i] >= 'a' && str[i] <= 'z') {
7.             str[i] = str[i] - 32; // Convert to Uppercase
8.         }
9.     }
10.    printf("Upper: %s", str);
11.    return 0;
12. }

```

خروجی برنامه:

```
1. Upper: HELLO
```

۱۹. پیدا کردن عنصر دوم بزرگ آرایه

بدون مرتب‌سازی کل آرایه می‌توانیم با یک بار پیمایش بزرگترین و دومین بزرگترین عدد را پیدا کنیم.

```

1. #include <stdio.h>
2.
3. int main() {
4.     int arr[] = {12, 35, 1, 10, 34, 1};
5.     int max1, max2;
6.     max1 = max2 = -1; // Assume values are positive
7.
8.     for(int i = 0; i < 6; i++) {
9.         if(arr[i] > max1) {
10.            max2 = max1;
11.            max1 = arr[i];
12.        } else if(arr[i] > max2 && arr[i] < max1) {
13.            max2 = arr[i];
14.        }
15.    }
16.    printf("Second largest: %d", max2);
17.    return 0;
18. }

```

خروجی برنامه:

```
1. Second largest: 34
```

۲۰. بررسی پالیندروم بودن رشته

بررسی می‌کنیم آیا یک کلمه مثل `madam` از دو طرف یکسان خوانده می‌شود یا خیر. از دو ایندکس شروع و پایان استفاده می‌کنیم.

```
1. #include <string.h>
2. #include <stdio.h>
3.
4. int main() {
5.     char str[] = "madam";
6.     int left = 0;
7.     int right = strlen(str) - 1;
8.     int isPal = 1;
9.
10.    while (left < right) {
11.        if (str[left] != str[right]) {
12.            isPal = 0;
13.            break;
14.        }
15.        left++;
16.        right--;
17.    }
18.
19.    if(isPal) printf("String is Palindrome");
20.    else printf("Not Palindrome");
21.    return 0;
22. }
```

خروجی برنامه:

1. String is Palindrome